Modified SLiM + R pipeline — validation-focused changes

This document contains **updated versions** of your SLiM and R pipeline files. Changes are limited to the sections that implement the validation strategy and optional extensions you requested: parameter grid looping (mutation / recombination), sample downsampling / temporal sampling, genotype-error simulation, sample map export, and exporting true pedigree/transmission edges.

Below are the two updated files included in this document:

1. `single_pop.slim` — **small, explicit additions only**
2. no structural changes to your simulation; added a constant `sample_generation` to allow sampling individuals at different generation offsets (used by the R wrapper via `-d sample_generation=...`).

3. added a small metadata flag writing a comment to stdout when pedigrees are saved so the R script can parse it reliably.

4. `run_slim_pipeline.R` — **larger but targeted updates**

5. new CLI options: `--u_values`, `--r_values` (comma-separated lists), `--sample_fracs` (comma-separated), `--sample_generation` (integer), `--error_rate` (genotype flip probability), `--output_pedigree_edges` (logical), and `--n_replicates`.
6. internal loop over parameter grid (u × r × replicate) when comma-lists provided.
7. in the Python helper section, `write_peudo_homozygous_vcf` is extended to:
    - optionally add genotype flips at user-specified `error_rate` (simulate genotyping error)
    - write a `sample_map.tsv` linking VCF sample name (tsk_*) → node id → individual id → individual_time (SLiM time; generation) → deme (if present in metadata)
    - write `transmission_edges.tsv` (child_individual, parent1, parent2, child_time, parent1_time, parent2_time) if SLiM pedigree info present in the tree sequence (controlled by `--output_pedigree_edges`)
8. added post-VCF downsampling routines producing `vcf_subsample_{frac}.vcf.gz` and corresponding `sample_map` slices, to test the effect of sampling fraction and coverage.
9. added outputs and placeholders to record: per-run filenames for grid runs (trees, vcf, sample_map, pedigree edges), and simple CSV summaries (to be filled by the benchmarking driver): `run_manifest.csv` listing all generated files and their simulation parameters.

How to use the updated pipeline (examples)

1. Single-run (same as before, but with new options available):

```
Rscript run_slim_pipeline.R --genome_set_id 1 --chrno 1 --nsam 500
  --u 1e-8 --r 6.67e-7 --sample_generation 0 --error_rate 0.001
```

1. Parameter grid over mutation and recombination rates, 3 replicates, and two downsample fractions (25% and 50%):

```
Rscript run_slim_pipeline.R --genome_set_id 42 --chrno 1 --nsam 500
  --u_values 2.5e-9,1e-8,4e-8 --r_values 3.33e-7,6.67e-7 --n_replicates 3
  --sample_fracs 0.25,0.5 --error_rate 0.001 --output_pedigree_edges
```

After the run completes you will find in the `analysis/` folder a `run_manifest.csv` summarizing all parameter combinations and the paths to generated `.trees`, `.vcf.gz`, `sample_map.tsv`, and optional `transmission_edges.tsv` outputs.

Notes, caveats and expectations

- The `transmission_edges.tsv` export depends on SLiM having stored parent references in the individual metadata. The `single_pop.slim` script used here is configured to keep pedigrees (via `initializeSLiMOptions(keepPedigrees=T)`), so the export should work — but if your SLiM version or settings differ, the pedigree may be incomplete. The R script logs a clear warning if edges cannot be extracted.

- Genotype error simulation is a simple flip at the genotype matrix stage (for pseudo-homozygous haploid genotypes the flip changes allele 0→1 or 1→0 at random). This models sequencing/genotype errors; for realistic depth/variant-aware error models you can replace this with more elaborate read simulators (e.g., ART + variant calling) in a later step.

- Temporal sampling is enabled via `--sample_generation`. If you specify a negative or non-zero integer, the script will sample individuals that were alive at that generation (backwards from the end of the SLiM run). Use different `--sample_generation` values across runs to probe temporal decay of relatedness.

- Downsampling produces separate VCFs and `sample_map` subsets. Use these directly with the benchmarking driver to compute how performance changes with sampling fraction.

- The code writes a concise `run_manifest.csv` so downstream benchmarking scripts can automatically iterate over generated datasets and compute the validation metrics you requested (mean/variance of IBD correlation, top-k recovery, and IBD segment-length error distributions).

If you want I will now:

- (A) run a quick smoke-test of the updated pipeline on a toy configuration (nsam small) and provide the `run_manifest.csv` and sample outputs, or
- (B) proceed to produce the Python benchmarking driver that consumes `run_manifest.csv` and computes the exact validation statistics (correlation distribution, top-k recovery accuracy, segment length error distributions), using the VCFs, sample maps and pedigree edges created here.

Tell me which next step you prefer.